

Generated 2026-03-02 23:29:47

WAFT Recent Work Report

- Generated: 2026-03-02 23:29:47
- Work effort root: /Users/ctavolazzi/Code/active/waft/_work_efforts
- Files reviewed: 12 work-effort artifacts + 10 devlog sections

Executive Snapshot

This report consolidates the latest indexed work efforts/checkpoints/session recaps and the newest devlog sections into one readable artifact.

Recent Work Efforts

Sitrep Hub Utility + Slash Command

- File: _work_efforts/WE-260302-wrpt_work_effort_reporting_slash_command/WE-260302-wrpt_index.md
- Status: completed
- Last Updated (frontmatter): 2026-03-02T23:35:00Z
- Last Modified (filesystem): 2026-03-02 23:29:19

Progress highlights: - 2026-03-02 23:20 PST: Created scripts/work_effort_report.py to aggregate recent work efforts + latest devlog sections. - 2026-03-02 23:20 PST: Implemented output generation for: - timestamped markdown (recent_work_report_*.md) - timestamped HTML (recent_work_report_*.html) - timestamped PDF (recent_work_report_*.pdf)

SCP Meme Discovery Dossier Generator

- File: `_work_efforts/WE-260302-scpd_scp_meme_discovery_dossier/WE-260302-scpd_index.md`
- Status: `active`
- Last Updated (frontmatter): `2026-03-03T07:16:00Z`
- Last Modified (filesystem): `2026-03-02 23:26:33`

Progress highlights: - 2026-03-02 06:40 PST: Work effort created for SCP meme discovery dossier generation. - 2026-03-02 06:41 PST: Added `scripts/generate_scp_meme_dossier.py` using WAFT `MemeGenerator` + `BriefDocument`. - 2026-03-02 06:41 PST: Script generated 3 meme artifacts and 1 SCP-style dossier PDF with embedded images and COI command traces. - 2026-03-02 06:42 PST: Validation complete; no linter errors in new script. - 2026-03-02 06:46 PST: Re-opened effort to deliver one unified webpage (React UI) for meme generation + dossier generation in the same interface.

Session Recap

- File: `_work_efforts/SESSION_RECAP_2026-03-02.md`
- Status: `n/a`
- Last Updated (frontmatter): `n/a`
- Last Modified (filesystem): `2026-03-02 14:22:04`

Excerpt:

```
# Session Recap

**Date**: 2026-03-02
**Time**: 14:22
**Timestamp**: 2026-03-02T14:22:01.197985

---

## Session Information

- **Date**: 2026-03-02 14:22
```

- **Branch**: feat/docker-ollama-runtime-github-update
- **Uncommitted Files**: 102

Accomplishments

- **Files Created**: 61
- **Lines Written**: 12,162

WAFT Meme Generator FFmpeg Appendage

- File: `_work_efforts/WE-260302-m3me_waft_meme_generator_ffmpeg_appendage/WE-260302-m3me_index.md`
- Status: `completed`
- Last Updated (frontmatter): `2026-03-02T14:38:15Z`
- Last Modified (filesystem): `2026-03-02 06:38:58`

Progress highlights: - 2026-03-02 06:35 PST: Work effort created and linked to existing meme planning context. - 2026-03-02 06:36 PST: Added `src/waft/core/meme_generator.py` with request models, style/template routing, Bouncer URL checks, topical fallback, and FFmpeg command assembly. - 2026-03-02 06:37 PST: Added `src/waft/cli/meme_cli.py`, wired `meme` command group in `src/waft/main.py`, exported core symbols, and updated allowlist hosts in `src/waft/config/port_manifest.example.json`. - 2026-03-02 06:37 PST: Added tests `tests/test_meme_generator.py` and `tests/test_meme_cli.py`. - 2026-03-02 06:38 PST: Added `docs/MEME_GENERATOR_GUIDE.md` and linked docs in `docs/DOCUMENTATION_INDEX.md` and `README.md`.

Work Effort: WAFT Ollama Runtime UI

- File: `_work_efforts/WE-260301-orui_ollama_runtime_ui/WE-260301-orui_index.md`
- Status: `n/a`
- Last Updated (frontmatter): `n/a`
- Last Modified (filesystem): `2026-03-02 06:29:35`

Progress highlights: - New work effort created per request for UI-specific runtime proof. - Added `GET /api/runtime-ui` in `src/waft/api/routes/ollama.py`. - UI includes: - Generate panel -> `POST /api/generate` - Chat panel -> `POST /api/chat`

Work Effort: WAFT Docker/Ollama Runtime

- File: `_work_efforts/WE-260301-wdor_waft_docker_ollama_runtime/WE-260301-wdor_index.md`
- Status: `n/a`
- Last Updated (frontmatter): `n/a`
- Last Modified (filesystem): `2026-03-01 20:04:09`

Progress highlights: - Created dedicated work effort for this implementation. - Selected v1 endpoint scope: `/api/generate` and `/api/tags` (defer `/api/chat`). - Confirmed container runtime default strategy: `uvicorn src.waft.api.main:app --host 0.0.0.0 --port <PORT>`. - Added `src/waft/api/routes/ollama.py` with Ollama-style `/api/tags` and `/api/generate`. - Wired new router in `src/waft/api/main.py`.

agentchatr_server_setup

- File: `_work_efforts/WE-260301-agct_agentchatr_server_setup/WE-260301-agct_index.md`
- Status: `completed`
- Last Updated (frontmatter): `2026-03-01T19:58:00.000Z`
- Last Modified (filesystem): `2026-03-01 19:57:55`

Excerpt:

```
---
id: WE-260301-agct
title: "agentchatr_server_setup"
status: completed
created: 2026-03-01T19:35:00.000Z
created_by: ctavolazzi
```

```
last_updated: 2026-03-01T19:58:00.000Z
repository: waft
---

# WE-260301-agct: Agentchattr Server-Only Setup

## Metadata
- **Created**: Sunday, March 1, 2026
- **Repository**: waft
- **Target Repo**: `https://github.com/bcurts/agentchattr.git`
- **Target Location**: `/Users/ctavolazzi/Code/active/agentchattr`
```

Oracle Deep Analyze Critique Workflow

- File: `_work_efforts/WE-260120-ebjt_oracle_deep_analyze_critique_workflow/WE-260120-ebjt_index.md`
- Status: `active`
- Last Updated (frontmatter): `2026-03-01T18:36:00.000Z`
- Last Modified (filesystem): `2026-03-01 18:31:42`

Progress highlights: - Created bananote research booklet and compiled PDF - Ran `waft oracle` (captured `NameError`) - Deep analyze + critique documents generated - Check-assumptions attempt logged with verification trace - Science-bitch run executed (context artifact captured)

Checkpoint: Cognitive Prosthetics, Oracle, and Doc Ingester

- File: `_work_efforts/CHECKPOINT_2026-03-01_cognitive_prosthetics_oracle_doc_ingester.md`
- Status: `n/a`
- Last Updated (frontmatter): `n/a`
- Last Modified (filesystem): `2026-03-01 17:17:01`

Excerpt:

```
# Checkpoint: Cognitive Prosthetics, Oracle, and Doc Ingester
```

****Date**:** 2026-03-01 17:16:39 PST
****Session**:** Cognitive prosthetics repo setup, oracle consult, and doc
****Status**:** 🚧 In Progress

Executive Summary

Established a new public upstream repository for cognitive prosthetics

Chat Recap

Conversation Summary

Handoff Brief: Agentchattr Server-Only Setup

- File: `_work_efforts/HANDOFF_BRIEF_2026-03-01_agentchattr_server_only.md`
- Status: `n/a`
- Last Updated (frontmatter): `n/a`
- Last Modified (filesystem): `2026-03-01 14:30:49`

Excerpt:

Handoff Brief: Agentchattr Server-Only Setup

****Date**:** 2026-03-01
****Work Effort**:** ``WE-260301-agct``
****Status**:** Complete (operational), hardening pending

1) What was done

- Created work effort and tracking artifacts for setup execution.
- Cloned ``agentchattr`` to ``/Users/ctavolazzi/Code/active/agentchattr``.
- Started server-only stack via ``sh macos-linux/start.sh``.
- Validated local services and endpoint reachability.

- Added recap, reflection, critique, and checkpoint artifacts.

2) Verified runtime state

- Web UI: `http://127.0.0.1:8300` (HTTP `200`)
- MCP HTTP: `http://127.0.0.1:8200/mcp` (reachable, probe returned `400`)

Checkpoint: Agentchattr Setup Recap + Reflect + Critique

- File: `_work_efforts/`
`CHECKPOINT_2026-03-01_agentchattr_server_setup_recap_reflect_critique`
- Status: `n/a`
- Last Updated (frontmatter): `n/a`
- Last Modified (filesystem): `2026-03-01 14:22:44`

Excerpt:

```
# Checkpoint: Agentchattr Setup Recap + Reflect + Critique

**Date**: 2026-03-01 14:21:22 PST
**Session**: Post-implementation documentation checkpoint
**Status**:  Complete

---

## Executive Summary

This checkpoint captures a full post-task pass over the completed `agen

---

## Chat Recap

### Conversation Summary

- You requested `/recap /reflect and /critique your work in a /checkpoi
```

Adversarial Critique: Agentchattr Server-Only Setup

- File: `_work_efforts/`
`CRITIQUE_2026-03-01_142122_agentchattr_server_setup.md`
- Status: `n/a`
- Last Updated (frontmatter): `n/a`
- Last Modified (filesystem): `2026-03-01 14:22:30`

Excerpt:

```
# Adversarial Critique: Agentchattr Server-Only Setup

**Date**: 2026-03-01
**Time**: 14:21:22 PST
**Target**: Completed server-only setup workflow for `agentchattr`
**Critique Mode**: Bad-faith / worst-case analysis

---

## Executive Summary

The implementation succeeded functionally, but from a hostile perspective

- **CRITICAL**: 0
- **HIGH**: 2
- **MEDIUM**: 4
- **LOW**: 3
```

Recent Devlog Sections

2026-03-02 - Local CLI Promotion Review Gate

```
Implemented a local WAFT CLI process for deciding whether candidate ch

### Development Plan

1. Add a dedicated CLI review command with explicit promotion gates.
```


2. Score readiness and produce an auditable markdown report artifact.
3. Wire command into main WAFT CLI and document usage.
4. Add focused tests for pass/fail review scenarios.

Execution Results

- Added new CLI module:
 - `src/waft/cli/promotion\_cli.py`
 - Command: `waft promote review`
- Review command behavior:
 - evaluates 5 gates: `scope`, `quality`, `docs`, `professional\_surface`
 - computes readiness score out of 10
 - supports optional test execution (`--run-tests`, `--test-command`)
 - writes markdown report to `\_work\_efforts/reports/promotion\_review\_\*.md`
 - exits `0` when promotion-ready, `1` otherwise
- Wired into main CLI:
 - `src/waft/main.py` via `app.add typer(..., name="promote")`
- Updated protocol doc with runnable command examples:
 - `docs/FOGSIFT\_WAFT\_PROMOTION\_PROTOCOL.md`
- Added tests:
 - `tests/test\_promotion\_cli.py`
 - Validation: `PYENV\_VERSION=3.14.3 python -m pytest tests/test\_promotion\_cli.py`

Status

- Request completed.

2026-03-02 - Promotion Review Auto-Demo + Chrome Visualization

Extended promotion tooling with a one-command demo that produces concrete

Development Plan

1. Add an automation script that constructs demo candidates and runs `waft promote review`
2. Produce human-readable artifacts (HTML + markdown reports).
3. Open demo artifacts in Chrome for immediate visual inspection.

4. Re-run focused tests.

Execution Results

- Added script:
 - `scripts/promotion\_review\_demo.py`
- Script behavior:
 - creates temporary git repos for:
 - promotion-ready candidate (src + tests + docs)
 - blocked candidate (includes internal `\_pyrite` path)
 - runs:
 - `python -m waft.main promote review ... --json`
 - writes:
 - `demo\_output/promotion\_review\_demo.html`
 - `demo\_output/promotion\_review\_pass.md`
 - `demo\_output/promotion\_review\_fail.md`
 - opens demo page and reports in Chrome (with browser fallback)
- Updated protocol doc:
 - `docs/FOGSIFT\_WAFT\_PROMOTION\_PROTOCOL.md` with Auto Demo command
- Live run:
 - `PYENV\_VERSION=3.14.3 python scripts/promotion\_review\_demo.py`
 - pass exit code: `0`
 - fail exit code: `1`
- Manual open:
 - `open -a "Google Chrome" demo\_output/promotion\_review\_demo.html`
- Validation:
 - `PYENV\_VERSION=3.14.3 python -m pytest tests/test\_promotion\_cli.py`
 - `3 passed`

Status

- Request completed.

2026-03-02 - Demo Result Rendering Fix

Fixed the promotion demo dashboard so pass/fail cards show real reading

Development Plan

1. Inspect why JSON extraction failed in demo.
2. Make CLI JSON emission parse-stable.
3. Regenerate demo and reopen in Chrome.

Execution Results

- Root cause:
 - ``console.print(json.dumps(...))`` wrapped JSON output across multiple lines
- Fixes:
 - updated ``src/waft/cli/promotion_cli.py`` to use plain ``print(json.dumps(...))``
 - updated git status collection to include file-level untracked entries
 - ``git status --porcelain --untracked-files=all``
- Validation:
 - ``PYENV_VERSION=3.14.3 python -m pytest tests/test_promotion_cli.py``
 - regenerated demo: ``PYENV_VERSION=3.14.3 python scripts/promotion_regen.py``
 - opened refreshed page in Chrome
- Result now shown in HTML:
 - pass candidate: ``Ready=True`, `Score=10/10``
 - fail candidate: ``Ready=False`, `Score=2/10``

Status

- Request completed.

2026-03-02 - Three-Case Promotion Demo (Pass / Borderline / Fail)

Extended the browser demo to include a middle "borderline" case so the

Development Plan

1. Add a borderline candidate generator to demo script.
2. Update demo HTML layout from 2 cards to 3 cards.
3. Regenerate artifacts and open in Chrome.

Execution Results

- Updated:
 - `scripts/promotion\_review\_demo.py`
 - Added `build\_borderline\_candidate(...)` (src + tests, no docs)
 - Added report output: `promotion\_review\_borderline.md`
 - Updated HTML to render pass/warn/fail cards (responsive grid)
- Updated docs:
 - `docs/FOGSIFT\_WAFT\_PROMOTION\_PROTOCOL.md` (demo now documented as t
- Live run:
 - `PYENV_VERSION=3.14.3 python scripts/promotion_review_demo.py --no-
 - pass exit: `0`
 - borderline exit: `1`
 - fail exit: `1`
- Validation:
 - `PYENV_VERSION=3.14.3 python -m pytest tests/test_promotion_cli.py
- Opened in Chrome:
 - `demo\_output/promotion\_review\_demo.html`

Status

- Request completed.

2026-03-02 - Gate-Level Reason Badges in Demo

Improved demo interpretability by showing explicit failed-gate labels

Development Plan

1. Include gate-level metadata in CLI JSON output.
2. Render failed-gate badges in demo HTML for each scenario.
3. Re-run demo/tests and reopen Chrome.

Execution Results

- Updated `src/waft/cli/promotion\_cli.py`:
 - `--json` now includes:

- `gate\_results`: structured list of all gate outcomes
- `failed\_gates`: concise list for UI display
- Updated `scripts/promotion\_review\_demo.py`:
 - removed unused import cleanup
 - added `render\_gate\_badges(...)`
 - cards now display:
 - pass: `all gates pass`
 - borderline: failed gate badge(s), e.g. `docs`
 - fail: failed gate badge(s), e.g. `professional\_surface`, `security`
- Validation:
 - `PYENV\_VERSION=3.14.3 python -m pytest tests/test\_promotion\_cli.py`
 - `PYENV\_VERSION=3.14.3 python scripts/promotion\_review\_demo.py --no-verify`
- Opened updated demo in Chrome:
 - `demo\_output/promotion\_review\_demo.html`

Status

- Request completed.

2026-03-02 - Oracle Consult + Another-Cycle Equivalent Run

Executed the requested Oracle consult and then ran the closest available

Development Plan

1. Run `waft oracle` with current promotion-gate initiative context.
2. Execute cycle-equivalent phases available in this CLI.
3. Handle missing command gaps with closest supported fallback.
4. Record outcomes and generated artifacts.

Execution Results

- Oracle:
 - Command: `PYENV\_VERSION=3.14.3 python -m waft.main oracle "Run a full cycle equivalent run with the current promotion-gate initiative context."`
 - Result: guidance returned `HALT` due to high uncertainty / low knowledge
- Cycle-equivalent commands executed:

```
- `check-assumptions`
- `analyze`
- `improve`
- `proceed`
- `reflect`
- `next-cmd`
- Command gap encountered:
  - `checkpoint` is not a current WAFT CLI command in this build.
  - Fallback executed: `PYENV_VERSION=3.14.3 python -m waft.main recap`
  - Artifact produced: `_work_efforts/SESSION_RECAP_2026-03-02.md`
- Notable generated outputs from run:
  - `_pyrite/analyze/analyze-2026-03-02-142135.md`
  - journal reflection entry appended by `reflect`
  - `_work_efforts/SESSION_RECAP_2026-03-02.md`

### Status

- Request completed with fallback for missing `checkpoint` command.
```

2026-03-02 - Meme Generator Buildout: History, Autoplay, Auto Demo

Built another substantial slice of the meme system to move it toward a

Development Plan

1. Add server-side history persistence and retrieval endpoint.
2. Add theater autoplay controls for generated meme galleries.
3. Add one-command demo seeding script to generate visible examples.
4. Add tests and run focused validation.
5. Open updated experience in Chrome.

Execution Results

- Updated `src/waft/api/routes/meme\_lab.py`:
 - Added JSONL history persistence:
 - `\_work\_efforts/reports/meme\_web\_artifacts/meme\_history.jsonl`

- Added helpers:
 - ``_history_file_path(...)``
 - ``_append_history_entry(...)``
 - ``_read_history_entries(...)``
- Added endpoint:
 - ``GET /api/meme-lab/history?limit=...``
- Added history logging for:
 - ``POST /api/meme-lab/generate-meme``
 - ``POST /api/meme-lab/cook-template/{template_name}``
- Extended UI behavior:
 - fetches server history on load
 - merges server + localStorage histories
 - theater autoplay toggle (``off|on``)
 - autoplay interval control (seconds)
- Added auto-demo script:
 - ``scripts/meme_lab_auto_demo.py``
 - Seeds three sample meme artifacts and prepends history entries
 - Prints run hints and can auto-open Chrome
- Updated docs:
 - ``docs/MEME_GENERATOR_GUIDE.md``
 - Added sections for autoplay/history and auto-demo usage
- Added API test coverage:
 - ``tests/api/test_meme_lab.py::test_history_endpoint_returns_recent_g``

Validation

- ``PYENV_VERSION=3.14.3 python -m pytest tests/api/test_meme_lab.py tes``
- Result: ``14 passed``
- Demo seed run:
 - ``PYENV_VERSION=3.14.3 python scripts/meme_lab_auto_demo.py --host 1``
- Chrome open:
 - ``open -a "Google Chrome" "http://127.0.0.1:8012/api/meme-lab"``

Status

- Request completed.

2026-03-02 - Readability Guardrails for Meme Text Rendering

Implemented hard renderer-level constraints so caption text remains readable

Development Plan

1. Add text fitting helper to wrap and clamp line count by target width
2. Normalize render canvas to known dimensions where needed.
3. Strengthen text legibility with box overlays and stroke settings.
4. Add tests that assert bounded rendering filter behavior.
5. Re-validate and open updated UI in Chrome.

Execution Results

- Updated `src/waft/core/meme\_generator.py`:
 - Added `\_fit\_text\_block(...)`:
 - wraps text based on estimated pixel width,
 - reduces font size when needed,
 - clamps to max lines with ellipsis fallback.
 - Updated `\_escape\_drawtext(...)` to be newline-safe.
 - Added normalized canvas preprocessing for top/band styles:
 - `scale=1280:720:force\_original\_aspect\_ratio=decrease`
 - `pad=1280:720:(ow-iw)/2:(oh-ih)/2:black`
 - Updated drawtext filters to use:
 - bounded center position (`x=max(20,(w-text\_w)/2)`),
 - readability boxes (`box=1:boxcolor=black@0.45:boxborderw=12`),
 - adaptive font sizes from fitted text blocks.
- Added tests in `tests/test\_meme\_generator.py`:
 - `test\_fit\_text\_block\_wraps\_and\_clamps\_lines`
 - `test\_ffmpeg\_filters\_include\_canvas\_normalization\_and\_box`
- Validation:
 - `PYENV\_VERSION=3.14.3 python -m pytest tests/test\_meme\_generator.py`
 - Result: `16 passed`
- Opened updated Meme Lab in Chrome:
 - `http://127.0.0.1:8012/api/meme-lab`

Status

- Request completed.

2026-03-02 - Core Meme Architecture Lock-In (Configurable CLI + Template Product Layer)

Delivered the two explicit architecture goals: a deeply configurable g

Development Plan

1. Expand template model into a catalog (featured quick-set + broader c
2. Upgrade CLI to expose all major generation/tuning parameters and con
3. Add template catalog API and UI browsing layer on top of existing sc
4. Improve determinism in template seeding.
5. Update tests and validate.

Execution Results

- Core template model expanded in `src/waft/core/meme\_generator.py`:
 - `MemeTemplate` now carries:
 - `category` (`mainstream`, `waft-native`)
 - `featured` (for quick-click soundboard)
 - Catalog expanded to include mainstream + WAFT-native formats.
 - Added `list\_featured\_templates()` for layered UX.
- CLI generation core upgraded in `src/waft/cli/meme\_cli.py`:
 - `waft meme generate` now supports:
 - `--temperature`
 - `--top-k`
 - `--creativity`
 - `--punchiness`
 - `--absurdity`
 - `--config` (JSON request override file)
 - Kept direct-call test compatibility with OptionInfo-safe fallback h
 - `waft meme templates` now displays category + featured metadata.
- Template product layer expanded in `src/waft/api/routes/meme\_lab.py`:
 - Added `GET /api/meme-lab/templates` for full catalog.

- Added UI "Template Browser" clickable section (mainstream + WAFT-na
- Soundboard remains featured quick-set (8 templates) from catalog me
- Updated template seed derivation to deterministic SHA-256 based met
- Docs updated:
 - `docs/MEME\_GENERATOR\_GUIDE.md` with full CLI tuning and template-la

Validation

- `PYENV_VERSION=3.14.3 python -m pytest tests/test_meme_cli.py tests/t
- Result: `22 passed`

Status

- Request completed.

2026-03-02 - Recent Work Report Utility + Global Slash Command

Added a one-command reporting utility that scans recent `\_work\_efforts`

Development Plan

1. Build a direct script that compiles recent work-effort and devlog co
2. Add formatted HTML and PDF generation with stable "latest" artifact
3. Add a slash command so the report is runnable by typing one command.
4. Sync the slash command to global Cursor commands.
5. Run and validate end-to-end output generation.

Execution Results

- Added script:
 - `scripts/work\_effort\_report.py`
- Script behavior:
 - reads recent work effort artifacts (`WE-\*/WE-\*\_index.md`, `CHECKPOI
 - reads recent `##` sections from `\_work\_efforts/devlog.md`
 - writes timestamped artifacts to `\_work\_efforts/reports/`:
 - `recent\_work\_report\_\*.md`

- `recent\_work\_report\_\*.html`
- `recent\_work\_report\_\*.pdf`
- writes stable aliases:
 - `recent\_work\_report\_latest.md`
 - `recent\_work\_report\_latest.html`
 - `recent\_work\_report\_latest.pdf`
- opens markdown/HTML/PDF artifacts in Chrome by default (use `--no-c`)
- Added slash command:
 - `./.cursor/commands/waft-report.md`
 - usage: `/waft-report`
- Synced commands globally:
 - `bash scripts/sync-cursor-commands.sh`
 - global command installed at `~/.cursor/commands/waft-report.md`
- Validation run:
 - `PYENV\_VERSION=3.14.3 python scripts/work\_effort\_report.py`
 - generated:
 - `\_work\_efforts/reports/recent\_work\_report\_20260302\_232058.md`
 - `\_work\_efforts/reports/recent\_work\_report\_20260302\_232058.html`
 - `\_work\_efforts/reports/recent\_work\_report\_20260302\_232058.pdf`
 - lints: no diagnostics in `scripts/work\_effort\_report.py`

Status

- Request completed.